

Paso 8 - Arreglos | Solucionario

Las seis soluciones del paso 8. Ojo con los índices: casi todos los errores de esta hoja son de **una unidad** (empezar en 1, terminar en `length`).

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso08` y `javac *.java`.

B8.1 - Las cinco notas

Un `for` llena el arreglo y otro (o un for-each) lo recorre para sacar las cuatro cuentas. Las cuatro se pueden hacer **en un solo recorrido**.

▼ Ver la solución (B8_1_CincoNotas.java)

B8_1_CincoNotas.java

```
import java.util.Arrays;
import java.util.Scanner;

public class B8_1_CincoNotas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double[] notas = new double[5];

        for (int i = 0; i < notas.length; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas[i] = sc.nextDouble();
        }

        double suma = 0;
        double mayor = notas[0];
        double menor = notas[0];
        int rojas = 0;

        for (double n : notas) {
            suma += n;
            if (n > mayor) {
                mayor = n;
            }
            if (n < menor) {
                menor = n;
            }
            if (n < 4.0) {
                rojas++;
            }
        }

        System.out.println();
        System.out.println("Notas: " + Arrays.toString(notas));
        System.out.printf("Promedio: %.1f%n", suma / notas.length);
        System.out.println("Nota mas alta: " + mayor);
        System.out.println("Nota mas baja: " + menor);
        System.out.println("Notas rojas: " + rojas);

        sc.close();
    }
}
```

```
}  
}
```

Salida:

```
Nota 1: 5.5  
Nota 2: 6.2  
Nota 3: 3.8  
Nota 4: 7.0  
Nota 5: 4.1  
  
Notas: [5.5, 6.2, 3.8, 7.0, 4.1]  
Promedio: 5.3  
Nota mas alta: 7.0  
Nota mas baja: 3.8  
Notas rojas: 1
```

Lo que importa. `mayor` y `menor` parten valiendo `notas[0]`, **no** 0 ni 7. Si `menor` partiera en 0, ninguna nota sería menor y siempre responderías 0; si `mayor` partiera en 0 funcionaría por casualidad, hasta que alguien use números negativos. Partir con el primer elemento es lo único que funciona siempre.

Fíjate en que las cuatro cuentas van en **un solo** `for`. Podrías escribir cuatro ciclos separados y estaría igual de correcto, pero acostumbrarse a hacerlo en uno es lo que después te permite escribir `informe()` en el paso 13 sin recorrer cinco veces lo mismo.

B8.2 - Al revés

Primero solo imprimir al revés (no toca el arreglo), después invertirlo de verdad (sí lo toca).

▼ Ver la solución (B8_2_Invertir.java)

B8_2_Invertir.java

```
import java.util.Arrays;

public class B8_2_Invertir {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30, 40, 50};
        System.out.println("Original: " + Arrays.toString(numeros));

        // 1. Solo imprimir al revés: el arreglo queda igual
        System.out.print("Impreso al revés: ");
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.print(numeros[i] + " ");
        }
        System.out.println();
        System.out.println("Sigue igual: " + Arrays.toString(numeros));

        // 2. Invertirlo de verdad: solo la MITAD de las vueltas
        for (int i = 0; i < numeros.length / 2; i++) {
            int auxiliar = numeros[i];
            numeros[i] = numeros[numeros.length - 1 - i];
            numeros[numeros.length - 1 - i] = auxiliar;
        }

        System.out.println("Invertido: " + Arrays.toString(numeros));
    }
}
```

Salida:

```
Original: [10, 20, 30, 40, 50]
Impreso al revés: 50 40 30 20 10
Sigue igual: [10, 20, 30, 40, 50]
Invertido: [50, 40, 30, 20, 10]
```

Por qué solo la mitad. Cada vuelta intercambia **dos** casilleros. Si recorrieras el arreglo completo, la segunda mitad volvería a intercambiar los mismos pares y el arreglo quedaría **como estaba**. Pruébalo: cambia `numeros.length / 2` por `numeros.length` y vas a ver el original otra vez.

El del medio no se toca cuando el largo es impar, y está bien: ya está en su lugar. Con 5 elementos, `5 / 2` da 2 (división entera del paso 3), o sea dos intercambios.

El intercambio con variable auxiliar es exactamente el B3.5. Aquí se ve para qué servía.

B8.3 - Buscar

Búsqueda lineal: recorrer hasta encontrar, guardando la posición, y `break` para no seguir buscando de más.

▼ Ver la solución (B8_3_Buscar.java)

B8_3_Buscar.java

```
import java.util.Scanner;

public class B8_3_Buscar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] numeros = {10, 20, 30, 40, 50, 60};

        System.out.print("Buscar: ");
        int buscado = sc.nextInt();

        int posicion = -1;
        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] == buscado) {
                posicion = i;
                break;
            }
        }

        if (posicion == -1) {
            System.out.println("No esta en la lista");
        } else {
            System.out.println("Encontrado en la posicion " + posicion
                               + " (el " + (posicion + 1) + "o)");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Buscar: 30
Encontrado en la posicion 2 (el 30)

--- ejecucion 2
Buscar: 99
No esta en la lista

--- ejecucion 3
Buscar: 10
Encontrado en la posicion 0 (el 10)
```

Por qué el `-1`. Necesitas un valor que signifique "no lo encontré", y no puede ser 0: el 0 es una **posición válida** (la primera). El `-1` es el acuerdo universal para esto, y lo vas a ver en `String.indexOf`, que devuelve `-1` cuando no encuentra lo que busca. Lo usas en el B11.4 para detectar el RUT sin guion.

Ojo con la diferencia entre **posición** e índice: el 30 está en el índice 2 y es el tercero. Al usuario se le habla de "el tercero"; adentro del programa todo es índice. Confundirlos es la causa del clásico error de una unidad.

B8.4 - El histograma

Un `for` recorre las notas y otro dibuja los asteriscos. La cantidad sale de un cast a `int`.

▼ Ver la solución (B8_4_Histograma.java)

B8_4_Histograma.java

```
public class B8_4_Histograma {  
    public static void main(String[] args) {  
        double[] notas = {5.5, 6.2, 3.8, 7.0, 4.1};  
  
        for (double nota : notas) {  
            System.out.printf("%.1f | ", nota);  
            for (int i = 0; i < (int) nota; i++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

Salida:

```
5.5 | *****  
6.2 | *****  
3.8 | ***  
7.0 | *****  
4.1 | ****
```

Lo que importa. El `(int) nota` corta el decimal: 5.5 da 5 asteriscos y 6.2 da 6. Si usaras `Math.round`, 5.5 daría 6 y 6.2 daría 6: los dos iguales, y el gráfico dejaría de distinguirlos. Para barras, **cortar** suele ser mejor que redondear.

Este ejercicio es el mismo `for` anidado del paso 7, pero con datos de verdad: el ciclo de afuera recorre un arreglo en vez de contar hasta 4.

B8.5 - La lista de ramos (ArrayList)

El menú del B6.3 con un `ArrayList` adentro. Las cinco operaciones son `add`, `size`, `get`, `remove` y `contains`.

▼ Ver la solución (B8_5_Ramos.java)

B8_5_Ramos.java

```

import java.util.ArrayList;
import java.util.Scanner;

public class B8_5_Ramos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> ramos = new ArrayList<>();
        int opcion;

        do {
            System.out.println();
            System.out.println("1) Agregar ramo");
            System.out.println("2) Listar ramos");
            System.out.println("3) Cuantos ramos");
            System.out.println("4) Borrar el primero");
            System.out.println("5) Buscar un ramo");
            System.out.println("0) Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();
            sc.nextLine(); // el Enter que dejo nextInt

            switch (opcion) {
                case 1:
                    System.out.print("Nombre del ramo: ");
                    ramos.add(sc.nextLine());
                    System.out.println("Agregado");
                    break;
                case 2:
                    for (int i = 0; i < ramos.size(); i++) {
                        System.out.println("  " + (i + 1) + ". " +
ramos.get(i));
                    }
                    break;
                case 3:
                    System.out.println("Tienes " + ramos.size() + " ramos");
                    break;
                case 4:
                    if (ramos.isEmpty()) {
                        System.out.println("No hay nada que borrar");
                    } else {
                        System.out.println("Borrado: " + ramos.remove(0));
                    }
                }
            }
        } while (opcion != 0);
    }
}

```

```

        }
        break;
    case 5:
        System.out.print("Que ramo buscas? ");
        String buscado = sc.nextLine();
        System.out.println(ramos.contains(buscado) ? "Si lo tienes"
: "No lo tienes");
        break;
    case 0:
        System.out.println("Chao");
        break;
    default:
        System.out.println("Esa opcion no existe");
    }
} while (opcion != 0);

sc.close();
}
}

```

Salida:

- 1) Agregar ramo
- 2) Listar ramos
- 3) Cuantos ramos
- 4) Borrar el primero
- 5) Buscar un ramo
- 0) Salir

Opcion: 1

Nombre del ramo: P00

Agregado

- 1) Agregar ramo
- 2) Listar ramos
- 3) Cuantos ramos
- 4) Borrar el primero
- 5) Buscar un ramo
- 0) Salir

Opcion: 1

Nombre del ramo: Base de datos

Agregado

- 1) Agregar ramo
- 2) Listar ramos
- 3) Cuantos ramos
- 4) Borrar el primero
- 5) Buscar un ramo
- 0) Salir

Opcion: 2

1. P00

2. Base de datos

- 1) Agregar ramo
- 2) Listar ramos
- 3) Cuantos ramos
- 4) Borrar el primero
- 5) Buscar un ramo
- 0) Salir

Opcion: 3

Tienes 2 ramos

- 1) Agregar ramo
- 2) Listar ramos
- 3) Cuantos ramos

```
4) Borrar el primero
5) Buscar un ramo
0) Salir
Opcion: 5
Que ramo buscas? P00
Si lo tienes
```

```
1) Agregar ramo
2) Listar ramos
3) Cuantos ramos
4) Borrar el primero
5) Buscar un ramo
0) Salir
Opcion: 4
Borrado: P00
```

```
1) Agregar ramo
2) Listar ramos
3) Cuantos ramos
4) Borrar el primero
5) Buscar un ramo
0) Salir
Opcion: 2
```

```
1. Base de datos
```

```
1) Agregar ramo
2) Listar ramos
3) Cuantos ramos
4) Borrar el primero
5) Buscar un ramo
0) Salir
Opcion: 0
Chao
```

Las respuestas del enunciado. Con un `String[]` habrías tenido que decidir **antes de empezar** cuántos ramos caben: `new String[10]` y rezar. Y para borrar el primero tendrías que correr todos los demás un casillero a la izquierda, a mano. El `ArrayList` hace eso solo: eso es todo lo que significa "crece".

El `sc.nextLine()` después del `nextInt()` vuelve a aparecer, y aquí es obligatorio: si no está, la opción 1 nunca te deja escribir el nombre del ramo.

El `? :` del caso 5 es el **operador ternario**: `condicion ? siVerdadero : siFalso`. Es un `if` que cabe en una expresión. Úsalo solo cuando quepa cómodo en una línea, como aquí.

B8.6 - Desafío: sin repetidos

Para cada número nuevo hay que recorrer **lo que ya llevas guardado**. Dos ciclos anidados, pero el de adentro crece de a poco.

▼ Ver la solución (B8_6_SinRepetidos.java)

B8_6_SinRepetidos.java

```
import java.util.Arrays;
import java.util.Scanner;

public class B8_6_SinRepetidos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] numeros = new int[5];
        int guardados = 0;

        while (guardados < numeros.length) {
            System.out.print("Numero " + (guardados + 1) + ": ");
            int numero = sc.nextInt();

            boolean repetido = false;
            for (int i = 0; i < guardados; i++) {        // solo lo ya guardado
                if (numeros[i] == numero) {
                    repetido = true;
                    break;
                }
            }

            if (repetido) {
                System.out.println("    El " + numero + " ya esta en la lista");
            } else {
                numeros[guardados] = numero;
                guardados++;
            }
        }

        System.out.println(Arrays.toString(numeros));
        sc.close();
    }
}
```

Salida:

```
Numero 1: 7
Numero 2: 3
Numero 3: 7
    El 7 ya esta en la lista
Numero 3: 9
Numero 4: 3
    El 3 ya esta en la lista
Numero 4: 1
Numero 5: 5
[7, 3, 9, 1, 5]
```

Lo que importa: el `while`, no un `for`. No sabes cuántas veces vas a pedir un número, porque los repetidos no cuentan. El contador `guardados` avanza **solo** cuando el dato sirve, y es a la vez la cantidad que llevas y el índice donde va el siguiente.

El `for (int i = 0; i < guardados; i++)` recorre hasta `guardados`, no hasta `numeros.length`. Los casilleros que todavía no llenaste valen 0, y si los comparas, un usuario que escriba 0 va a recibir "ya está en la lista" sin haberlo escrito nunca.

Este ejercicio es, en chiquitito, lo que hace un `Set` en Java: una colección sin repetidos. Lo vas a ver más adelante en la carrera; por ahora, lo escribiste tú.

Volver a la [hoja del paso 8](#) o al [paso 8 de la guía](#).